

AURUMCHAIN

Technical Documentation Pack

System Architecture • Smart Contract Specification • API Specification • User & Admin Flows • Custody & Authority Control • Security & Testing Plan

1. Executive Summary

The platform is a Solana-based tokenized investment MVP where investors complete KYC/AML, connect a supported Solana wallet, invest in approved projects using USDC or USDT, receive a project-specific SPL token representing fractional ownership, and receive profit distributions proportional to token holdings.

The solution must be secure, compliant, auditable, and scalable. The mandatory scope requires API-based KYC/AML verification, multi-signature treasury custody, restricted token transfers to verified wallets only, a backend transaction ledger, full audit trail, client-controlled deployment wallets, admin controls, and future-readiness for a secondary marketplace and other Phase 2 capabilities.

This pack turns that scope into a concrete technical design package suitable for a proposal appendix, funding review, or milestone annex.

1.1 Scope Summary

Included in MVP
Registration and authentication KYC/AML via API Wallet integration Stablecoin investment Project token allocation Portfolio dashboard Profit distribution Admin panel

Implementation Note:

The architecture, technical specifications, and flows outlined in this document represent the intended system design for the MVP. However, they are subject to refinement and change during the active development stage based on technical feasibility, security considerations, and implementation discoveries.

2. System Architecture Document

2.1 Architecture Principles

- Security-first financial architecture with strong separation between application logic, custody controls, compliance enforcement, and blockchain execution.
- Modular service boundaries so the MVP can expand into a marketplace, transfer features, and additional financial products without redesigning the core platform.
- Client ownership and operational control from deployment onward: treasury custody, deployment wallets, and upgrade authority must not remain solely with the vendor.

2.2 Logical Architecture

- Presentation Layer: Next.js web application with investor dashboard and admin dashboard.
- Identity & Compliance Layer: registration, authentication, Sumsb integration, sanctions / AML status sync, wallet-to-user identity binding, RBAC, and session handling.
- Application Layer: project catalog, investment workflow, order validation, subscription processing, ledgering, profit distribution orchestration, admin workflows, reporting, and notifications.
- Blockchain Layer: Solana programs, SPL token minting / allocation, restricted transfers, on-chain profit distribution, and transaction verification via Solana Web3.js.
- Data Layer: PostgreSQL for structured operational data, MongoDB for flexible or semi-structured records, and immutable blockchain transaction references stored in the ledger.
- Infrastructure Layer: AWS (or equivalent), Dockerized services, CI/CD, secrets management, logging, monitoring, and production security hardening.

Logic Split Breakdown:

Backend Responsibilities: Managing the user session, handling KYC/AML webhooks from Sumsb storing off-chain investment ledgers, validating investment thresholds, and calculating the mathematical entitlements for profit distributions off-chain.

On-chain Responsibilities: SPL token minting, enforcing transfer restrictions to verified wallets only holding the immutable project state (supply cap, mint address), and executing the actual transfer of USDC/USDT for profit payouts.

2.3 End-to-End Data Flow

- The user signs up and completes KYC/AML. Backend receives verification updates from Sumsb and marks the user as approved or restricted.
- The user connects a supported Solana wallet. Backend links the wallet address to the verified identity record and updates allow-list eligibility.
- The user selects a project, enters an investment amount, and the backend validates user status, thresholds, lock-up rules, and treasury routing rules.
- Investor transfers USDC/USDT to the multi-signature treasury route; backend records pending investment and verifies on-chain settlement.
- After settlement, the system allocates the corresponding project SPL tokens to the verified wallet and updates both the backend ledger and on-chain references.
- When profits are declared, admin submits the payable amount, backend prepares the distribution batch, Solana program executes proportional payouts, and all results are logged for reconciliation and investor visibility.

2.4 Proposed Component Map

- API gateway / backend (Node.js / Express): auth, project APIs, investments, distributions, admin operations, webhooks, internal services.
- Compliance service: Sumsb integration client, webhook processing, status sync, sanctions / AML storage, re-verification rules.
- Wallet service: wallet-link verification, address challenge signing, allow-list sync, transaction status monitoring.
- Tokenization service: project configuration, mint orchestration, token allocation, lock-up / restriction policy evaluation.
- Distribution service: entitlement calculation, batch creation, payout instruction generation, reconciliation, reporting.
- Ledger & reporting service: immutable internal event log for investments, token allocations, payouts, exceptions, and audit exports.
- Monitoring stack: structured logs, metrics, alerting, incident hooks, and chain health dashboards.

2.5 Architecture Flow Diagram (Textual)

Investor Web App -> API Gateway -> Auth / Compliance Service -> PostgreSQL / MongoDB

Investor Web App -> Wallet Adapter -> Solana Network

API Gateway -> Tokenization / Distribution Services -> Solana Programs

API Gateway -> Treasury / Transaction Monitor -> Multi-Signature Treasury

Admin Web App -> API Gateway -> Project / Distribution / Control Services

Webhook Providers (Sumsb) -> Compliance Service -> User Eligibility State

2.6 Sample Database Schema Outline

users is the root entity. Every investor or admin account lives here. It stores credentials, role (**investor**, **admin**), and account status. All other entities trace back to this table via **user_id**.

sessions manages JWT and refresh token lifecycle. Tokens are stored as hashes only — never plaintext. Expiry and IP are tracked for security auditing.

kyc_records holds one or more KYC/AML records per user (a new record is created on re-verification). It stores the Sumsb **applicant_id**, individual statuses for KYC, AML, and sanctions, rejection reasons, and the raw webhook response payload as **jsonb** for auditability.

user_wallets links Solana wallet addresses to users via signed-message challenge. A user can link multiple wallets. Stores the chain, link status, and the nonce used for the on-chain challenge.

wallet_eligibility is the derived allow-list record — it joins a wallet to a KYC record and stores explicit **investment_allowed** and **transfer_allowed** flags with validity windows. The smart contract's transfer-restriction program checks against this.

projects is the project catalog. Each record holds all investment parameters: supply cap, min/max thresholds, accepted stablecoin, the treasury wallet address, lock-up period, and subscription window.

project_tokens is a one-to-one extension of **projects** storing the on-chain SPL token details: mint address, token name and symbol, decimals, program ID, and mint authority revocation status.

investments is the subscription ledger. Each record tracks a single investor's investment in a project: the amount, stablecoin used, the on-chain settlement transaction hash, and the resulting token allocation amount. Status moves through **pending** → **settled** → **allocated**.

token_allocations records the on-chain token delivery event per investment. It stores the allocation transaction hash and links back to the specific wallet that received the tokens.

distribution_epochs represents each profit distribution event. It captures the total profit amount, the record date (snapshot point), execution status, and which admin triggered it.

distribution_payouts is one row per investor per epoch. It stores the entitlement calculation, the token holding snapshot at record date, the payout transaction hash, and individual settlement status.

audit_log is an immutable append-only event log. Every significant action (KYC status change, wallet link, investment creation, admin action, pause toggle) writes a row here with before/after state snapshots.

MongoDB — Flexible / Semi-Structured Collections

These complement for data that is schema-variable or high-volume:

Collection	Purpose
<code>kyc_webhook_events</code>	Raw inbound Sumsbub webhook payloads, preserved exactly as received for replay and debugging
<code>notification_log</code>	Email/SMS delivery records, templates used, delivery status
<code>project_documents</code>	Rich project content: offering documents, risk disclosures, attachments (variable structure per project)
<code>admin_activity_feed</code>	High-frequency admin UI activity stream, useful for dashboards without burdening database.
<code>chain_transaction_cache</code>	Cached Solana transaction confirmations to reduce RPC calls; keyed by tx hash
<code>investor_portal_sessions</code>	Extended session metadata, device info, and UI preference state

3. Smart Contract Specification

Proposed implementation detail: the source scope mandates controlled minting, ownership tracking, verified-wallet transfer restrictions, whitelisting, and proportional profit distribution. The following program split is a practical design to satisfy those requirements.

3.1 Program Set

Item	Specification
Project Registry Program	Stores project metadata, project status, mint references, supply configuration, min/max investment thresholds, lock-up settings, and distribution cadence.
Compliance / Transfer Control Program	Validates whether a wallet is linked to a verified investor and whether a transfer is permitted. Can be

Item	Specification
	implemented via program-controlled transfers or a Token-2022 compatible restriction model.
Allocation & Distribution Program	Handles token issuance records, subscription finalization, distribution snapshots, entitlement calculations, and payout execution rules.

3.2 Token Rules

Item	Specification
Token standard	Dedicated SPL token per project representing fractional ownership.
Supply control	Token supply defined per project. Minting allowed only through authorized program flow.
Mint authority	Must be revoked after configured issuance or controlled by client-approved multi-signature.
Burning	Permitted only under explicit admin and policy-controlled events such as correction, redemption, or cancellation according to platform rules.
Transfer restriction	Only approved wallets may receive or transfer tokens; unverified or sanctioned wallets are blocked.
Lock-up	Transfers may be blocked during project-specific mandatory lock-up periods.

3.3 Required Accounts / State

Item	Specification
Project account	Project ID, mint address, supply cap, subscription window, thresholds, lock-up, treasury references, active / paused state.
Investor eligibility account	Wallet address, identity reference hash, KYC status, AML / sanctions state, approval timestamp, re-verification state.
Investment subscription account	Investor, project, amount, payment asset, status, settlement transaction hash, allocated token amount.
Distribution epoch account	Project, record date, total profit amount, payable asset, entitlement calculation state, payout batch status.
Control account	Pause flags, authority addresses, upgrade status, and operational limits.

3.4 Core Instructions

Item	Specification	Inputs	Outputs
create_project	Create project parameters, assign mint, supply cap, thresholds, and state.	projectId , supplyCap , minInvestment, maxInvestment, lockupTimestamp,	ProjectCreated.
update_project_status	Pause or unpause subscriptions and project operations.	projectId, isActive , statusFlags	PauseStateChanged
record_verified_wallet	Register or refresh verified-wallet eligibility.	walletAddress (Pubkey) ,identityHash, kycStatus expiryTimestamp	WalletVerified
subscribe_investment	Validate project rules and create a pending investment record.	projectId, investmentAmount, paymentAsset	subscriptionId
finalize_subscription	Confirm treasury settlement and allocate tokens.	subscriptionId, settlementTxHash, allocatedTokens	InvestmentSettled, TokensAllocated
transfer_validate	Authorize or reject token movement based on KYC and lock-up state.	Sender, receiver, amount, projectId	Returns Boolean (Allowed/Rejected)
create_distribution	Create payout epoch and freeze entitlement snapshot.	projectId, totalProfitAmount, payableAsset,recordDate	DistributionCreated
execute_distribution	Distribute profit proportionally to current eligible token holders.	epochId, investorWallet, entitlementAmount	DistributionExecuted
burn_or_adjust	Admin-governed correction path with full audit trace.	targetWallet,amount, reasonCode, auditHash	TokensBurned or BalanceAdjusted

3.5 Events and Auditability

Item	Specification
ProjectCreated	New project initialized.

Item	Specification
WalletVerified	Wallet linked or status refreshed.
InvestmentSettled	Stablecoin payment verified and subscription confirmed.
TokensAllocated	Investors received project token allocation.
DistributionCreated	Profit distribution epoch created.
DistributionExecuted	Payout transaction completed.
PauseStateChanged	Investments or transfers paused / resumed.

3.6 Non-Functional Requirements

Item	Specification
Security	Program logic written in Rust with strict account validation, signer checks, overflow-safe math, replay protection, and explicit authority checks.
Audit readiness	Clear account schema, deterministic calculations, error-code catalog, test coverage, and third-party audit preparation.
Upgrade path	Program upgrade authority transferred to client after deployment; upgrade process documented and multi-signature controlled.

4. API Specification

The backend is a REST API over Node.js / Express with JWT-based authentication for authenticated routes, role-based authorization for admin functions, secure webhook handling for compliance updates, and transaction logging for all investment-critical actions.

Authentication & Identity

Method	Endpoint	Purpose
POST	/auth/register	Create an investor account and begin onboarding.
POST	/auth/login	Authenticate user and issue JWT / refresh token.
POST	/auth/logout	Invalidate active session.
GET	/me	Return current user profile, KYC state, linked wallets, and permissions.

KYC / AML & Wallet

Method	Endpoint	Purpose
POST	/kyc/session	Create a Sumsb applicant / session.
POST	/webhooks/sumsub	Receive verification status updates.
GET	/kyc/status	Fetch current KYC/AML status for logged-in user.
POST	/wallets/link	Link wallet to user through signed message challenge.
GET	/wallets	List linked wallets and eligibility state.
POST	/wallets/reverify	Trigger wallet re-verification flow.

Projects & Investments

Method	Endpoint	Purpose
GET	/projects	List available projects and public investment parameters.
GET	/projects/{projectId}	Return project detail, thresholds, status, lock-up, and token information.
POST	/investments/quote	Calculate expected allocation and validate thresholds.
POST	/investments/subscribe	Create pending subscriptions.
POST	/investments/confirm	Confirm on-chain stablecoin settlement and finalize allocation.
GET	/portfolio	Return holdings, valuations, pending subscriptions, and history.
GET	/transactions	Return investor transaction and distribution history.

Distributions

Method	Endpoint	Purpose
GET	/distributions	List distributions visible to the current investor.
GET	/distributions/{distributionId}	Return entitlement, payout status, and chain references.

Admin

Method	Endpoint	Purpose
POST	/admin/projects	Create a project and configure issuance parameters.
PATCH	/admin/projects/{projectId}	Update project parameters or status.
POST	/admin/distributions	Create a profit distribution batch.
POST	/admin/distributions/{distributionId}/execute	Trigger on-chain payout execution.
POST	/admin/controls/pause-investments	Pause or resume subscription activity.
POST	/admin/controls/pause-transfers	Pause or resume token transfer operations.
GET	/admin/investors	List investors, KYC state, wallet state, and limits.
GET	/admin/audit-log	Return immutable operational audit events.

5. User Flow Diagrams

5.1 Investor Onboarding Flow

1. Open platform
2. Register account
3. Verify email / phone (if enabled)
4. Start KYC session
5. Submit identity documents
6. Receive approval / rejection / pending state
7. Link Solana wallet by signed message
8. Wallet becomes eligible for investment once KYC and AML checks pass

5.2 Investment Flow

1. Browse project catalog
2. Open project detail
3. Review thresholds, lock-up, risk text, and token terms
4. Enter investment amount
5. System validates min / max thresholds and eligibility
6. User confirms payment
7. Stablecoin transfer sent to treasury route

8. Backend verifies settlement
9. SPL tokens allocated to verified wallet
10. Portfolio updates with investment and transaction history

5.3 Distribution Flow

1. Investor holds project tokens
2. Admin declares distributable profit
3. System calculates token-holder entitlements
4. On-chain distribution is executed
5. Investor sees payout status and history in dashboard

6. Admin Workflow Diagrams

6.1 Project Setup Workflow

1. Create project record
2. Define project supply, thresholds, lock-up, and distribution cadence
3. Assign or initialize project mint
4. Publish project as active
5. Monitor subscriptions and treasury receipts

6.2 Distribution Workflow

1. Review project performance / payable amount
2. Enter profit amount in admin backend
3. Generate entitlement snapshot
4. Review batch and approve execution
5. Trigger on-chain payout distribution
6. Reconcile payout results and publish history to investors

6.3 Emergency Control Workflow

1. Detect issue or receive compliance / security trigger
2. Pause investments or transfers
3. Investigate incident and capture audit notes
4. Apply remediation or re-verification
5. Resume operations after approval

7. Custody and Authority-Control Design

7.1 Custody Model

- Investor subscription funds are routed only to a multi-signature treasury wallet with no unilateral control by any one party.
- Recommended treasury model: 2-of-3 or 3-of-5 client-controlled signature set depending on governance preference, with one or more client signers and one operational signer for execution support.
- The backend never has direct custody of assets; it orchestrates records, validations, and monitoring only.

7.2 Authority Matrix

- Treasury wallet authority: client multi-signature.
- Token mint authority: revoked after issuance window or retained under client multi-signature only.
- Program upgrade authority: transferred to client after deployment.
- Deployment authority: all deployments executed from client-controlled wallets.
- Pause authorities: restricted admin role with multi-party approval policy for production.

Admin Roles & Permissions:

Super Admin (Client Multisig): Exclusive authority over treasury fund withdrawals , smart contract upgrades , and assigning/revoking mint authority.

Operational Admin: Can create projects via /admin/projects , update public project metadata, and trigger emergency /admin/controls/pause-investments. **Cannot move funds.**

7.3 Operational Controls

- All treasury movement requests require explicit signing flow outside normal backend runtime.
- All authority changes are recorded in an operational change log and linked to governance approvals.
- Emergency actions are limited to pause functions and must not allow arbitrary withdrawal by the vendor.

7.4 Recommended Environment Separation

- Separate Devnet, staging, and Mainnet wallets, treasury addresses, and key material.
- Use hardware wallets or institution-grade signer custody for production signatures.
- Store non-custodial service secrets in a managed secrets vault with rotation policy.

7.5 Authority-Control Table

Control Surface	Required Owner	Rule	Evidence / Audit Output
Treasury wallet	Client multisig	No unilateral fund control	Signer roster, wallet config, treasury tx history
Mint authority	Revoked or client multisig	Controlled issuance only	Mint authority record, governance approval
Program upgrade	Client	Vendor cannot retain power after upgrade handover	Upgrade authority transfer tx

Deployment wallet	Client	All production deployments use client-controlled wallets	Deployment transaction receipts
Pause functions	Restricted admin authority	Emergency-only operational control	Pause event log, incident record

8. Security and Testing Plan

8.1 Security Controls

Area	Plan
Identity & access	JWT authentication, secure session handling, RBAC, segregated admin permissions, and production-grade password policies.
Data security	Encryption at rest for databases and secrets, TLS for all API traffic, minimized PII storage, and retention controls.
Blockchain security	Rust-based secure program development, authority validation, allow-list enforcement, pause controls, and audit-friendly code structure.
Operational security	Secrets management, CI/CD controls, least-privilege cloud access, immutable logging, alerting, and environment separation.
Compliance security	Automated KYC / AML status synchronization, sanctions screening, re-verification flows, and wallet-to-identity binding.

8.2 Test Strategy

Area	Plan
Unit testing	Backend business rules, entitlement calculations, validation functions, and serialization / parsing logic.
Integration testing	API-to-database, API-to-Sumsub, wallet-link flow, on-chain monitoring, and payout orchestration.
Smart contract testing	Program instruction tests, authority checks, invalid account tests, transfer restriction tests, and distribution math.
End-to-end testing	Onboarding, KYC pass/fail, wallet link, investment, allocation, portfolio visibility, admin payout, and pause flows.
Resilience testing	Failed transactions, network delays, duplicate webhook delivery, retry logic, and reconciliation integrity.

Security testing	Access-control tests, secrets exposure checks, dependency scanning, static analysis, and pre-deployment review.
------------------	---

8.3 Exit / Acceptance Checklist

- ☐ All critical user and admin flows tested successfully on target environment.
- ☐ Smart contract tests pass including restriction and authority scenarios.
- ☐ KYC / AML sync verified end to end with webhook handling.
- ☐ Treasury and authority configuration verified under client control.
- ☐ Audit logs, transaction history, and reconciliation exports available.
- ☐ Pause controls tested and operational runbook documented.
- ☐ Devnet to Mainnet deployment process completed and signed off.

8.4 Deployment Flow

1. **Devnet:** Contracts deployed to Solana Devnet. Backend connected to mock USDC/USDT and the client-provided Sumsb API (Sandbox environment). Used for internal QA, unit testing, and initial frontend integration.
2. **Staging / Testnet:** Contracts deployed to Solana Devnet/Testnet. Backend connected to the staging environment, continuing to utilize the client-provisioned Sumsb API. Used for Client Acceptance Testing (UAT).
3. **Mainnet (Production MVP Launch):** Final deployment of the MVP to Solana Mainnet. This phase is designed to accommodate execution via client-provisioned deployment wallets, establishing direct client ownership of the MVP infrastructure from day one if preferred

9. Deliverable Mapping to Client Scope

Requested Item	Where Covered	Status
System architecture document	Section 2	Included
Smart contract specification	Section 3	Included
API specification	Section 4	Included
User flow diagrams	Section 5	Included
Admin workflow diagrams	Section 6	Included
Custody and authority-control design	Section 7	Included
Security and testing plan	Section 8	Included

End of document.